

### Structural Schematic (Generated via DigitalJS Extension):



| time(ps) | clk | D | Q | expQ   | note |
|----------|-----|---|---|--------|------|
| 6        | 1   | 1 | 1 | (init) |      |
| 16       | 1   | 1 | 1 | PASS   |      |
| 26       | 1   | 0 | 1 | PASS   |      |
| 36       | 1   | 1 | 1 | PASS   |      |
| 46       | 1   | 0 | 0 | PASS   |      |

For Latch (level sensitive):  $Q = (Clk * D) + (\sim Clk * Q_{prev})$

**Explanation:** At power up, outputs can be undefined until a valid input or reset drives them. I used the two D-latches, which are level-sensitive to create an edge-triggered D-Flip-Flop. Each latch is comprised of four NAND gates:

- The first two NAND gates will take the inputs (Clk, D) and ( $\sim$ Clk, D)
- The second two NAND gates each take an input from one of the first two NANDs and then each other's outputs, making the last two gates cross-coupled. This cross-coupled feedback is where the memory of the previous output comes from.

For NAND gates, 0 is the controlling value (output will always be 1) and 1 is the noncontrolling value (output depends on the other input/ is “transparent”). When Clk=0, the latch closes and holds, and D becomes a “don’t care” because for any value of D, the output will still be 1. When

Clk=1, D gets to control the output. This makes the latch triggered by the high level and hold at the low level. As for the D-flip-flop, we need two latches in series: one master and one slave:

- Master Latch:
  - Captures D when  $\sim\text{Clk}=1 \rightarrow$  output D
- Slave Latch:
  - Passes master's value when  $\text{Clk}=1 \rightarrow$  final Output

Combined, the output only changes once per edge, making the D-flip-flop a posedge-triggered storage.

### Waveform Test (Structural):



### Waveform Test (Behavioral):



## Design Code (Structural):

```
/*
4-NAND circuit with...
    2 NANDs gating (D*clk), (D*nclk)
    2 NANDs making cross-coupled SR latch --> stores bit
RESULTS IN LEVEL SENSITIVE D latch:
    transparent when clk=1, holds when clk=0
    --> Q follows when D wiggles while clk=1
*/

`timescale 1ps/1ps
//reg, assign only used in behavioral verilog
//NAND = ~(A&B)
module NAND(input a, b, output y);
wire temp;
and a0(temp, a, b);
not n0(y, temp);
endmodule

//Q=w4, Qb=W3
module D_latch_structural(input D, clk, output Q, Qb);
wire w1, w2;
wire nD;
not (nD, D);

//user defined modules need names, but built in primitives don't
NAND u0(.a(clk), .b(D), .y(w1));
NAND u1(.a(clk), .b(nD), .y(w2));
NAND u2(.a(w1), .b(Qb), .y(Q));
NAND u3(.a(w2), .b(Q), .y(Qb));

endmodule

/*
D flipflop = 2 latches in series w complementary enables
    Master: enabled when clk=0 (captures D)
    Slave: enabled when clk=1 (outputs master output)
*/

`timescale 1ps/1ps
```

```

//Q=w4, Qb=W3
module D_FF_structural(input D, clk, output Q, Qb);
wire clkn;
//need wire for not clk for slave (second circuit)
not(clkn, clk);

wire Qm, Qmb; //master latch outputs
//master latch on when clk=0, D=x -> 0 is noncontrolling for NAND
D_latch_structural master(.D(D), .clk(clkn), .Q(Qm), .Qb(Qmb));
D_latch_structural slave(.D(Qm), .clk(clk), .Q(Q), .Qb(Qb));

endmodule

```

### Design Code (Behavioral):

```

//technically not needed for D_FF_behaviorial
`timescale 1ps/1ps

//signal assigned inside always/ initial block must be reg
module D_latch_behavioral(input D, clk, output reg Q, Qb);

//update Q only when clk is high
//transparent when clk=1, blocking when clk=0
// --> REFLECTS CHANGES IN D
//<= is nonblocking assignment
always @ (D or clk) begin
    //if clk high, val in D assigned to Q, Q=D
    //for nonblocking <= RHS evaluated, then updates to LHS
    // AT THE END OF CURRENT SIM TIME STEP
    //nonblocking assignments SCHEDULED to happen LATER
    // USING initial VAL of RHS at beginning of time step
    if(clk) Q <= D;
    //else : hold (no assignment)
end
assign Qb = ~Q;

endmodule

```

```

`timescale 1ps/1ps
`default_nettype wire

```

```

//Q=w4, Qb=w3
module D_FF_behavioral(input D, clk, output reg Q, Qb);

//rising edge triggered register (transparent/ dont care when clk=1)
always @ (posedge clk) begin
    Q <= D;
end

assign Qb = ~Q;
endmodule

```

### Testbench Code (Structural):

```

`timescale 1ps/1ps

module structural_tb;
    reg clk=1'b0;
    reg D=1'b0;
    wire Q, Qb;

    D_FF_structural dut(.D(D), .clk(clk), .Q(Q), .Qb(Qb));

    //always blocks execute forever
    //they keep looping throughout the simulation
    //--> clk toggled every 5 ps
    always #5 clk = ~clk; //toggles at t=5 ps
    //intial blocks execute 1 time only
    initial begin
        $dumpfile("structural_wave.vcd");
        $dumpvars(0, structural_tb);
        #1  D = 0;    // before first rising edge
        #3  D = 1;    // still before first rising edge (clk low)
        #6  D = 0;    // during clk high
        #4  D = 1;    // now at t=10 (clk low)
        #3  D = 0;    // t=13 (clk low, near rising at 15)
        #5  D = 1;    // t=18 (clk high)
        #4  D = 0;    // t=22 (clk low)
        #6  D = 1;    // t=28 (clk high)
        #7  D = 0;    // t=35 (exactly at rising edge next line will check)
        #8  $finish;
    end
endmodule

```

```

end

// Scoreboard / checker
integer pass=0, fail=0;
reg expQ;          // expected Q (register model)
reg initialized=0; // ignore the very first edge if Q starts X

initial expQ = 1'b0;

// Update expected model at rising edges (posedge-triggered FF)
always @(posedge clk) begin
    expQ <= D;
end

// Compare a tiny time after each rising edge (let gates settle)
always @(posedge clk) begin
    #1; // small settle time to avoid delta-cycle races
    if (!initialized) begin
        initialized = 1;
        $display("time(ps)  clk  D  Q  expQ  note");
        $display("%8t   %0d   %0d  %0d  %0d   (init)", $time, clk, D, Q, expQ);
    end else if (Q === expQ) begin
        pass = pass + 1;
        $display("%8t   %0d   %0d  %0d  %0d   PASS", $time, clk, D, Q, expQ);
    end else begin
        fail = fail + 1;
        $display("%8t   %0d   %0d  %0d  %0d   **FAIL**", $time, clk, D, Q, expQ);
    end
end

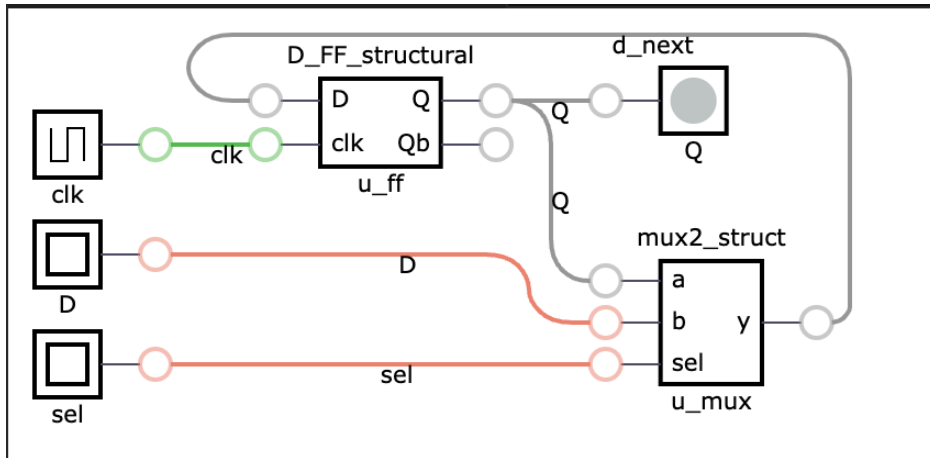
// Final summary
initial begin
    #60; // ensure summary prints even if finish moves
    $display("\nSUMMARY: pass=%0d fail=%0d", pass, fail);
end

endmodule

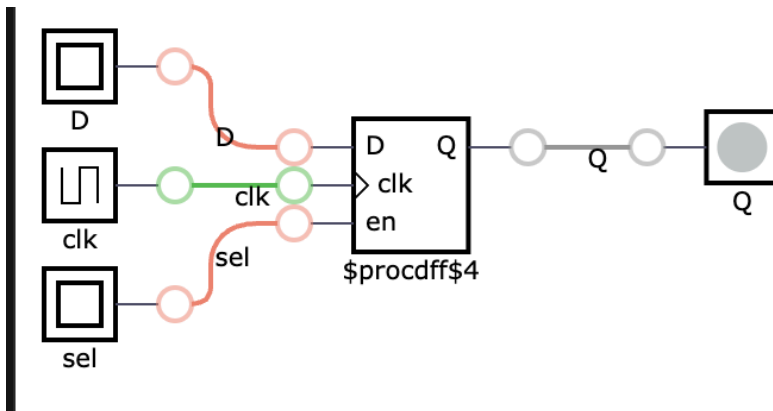
```

## Part 2: One-Bit Register

Structural Schematic (Generated via DigitalJS Extension):



Behavioral Schematic (Generated via DigitalJS Extension):



Truth Table:

0 ns | start (Q may be x until first edge) | clk=0 sel=0 D=0 -> Q=x  
 10000 ns | prep: load 1 | clk=0 sel=1 D=1 -> Q=x  
 15000 ns | after clk↑ (Q should be 1) | clk=1 sel=1 D=1 -> Q=x  
 20000 ns | prep: hold, D=0 | clk=0 sel=0 D=0 -> Q=1  
 25000 ns | after clk↑ (Q should still be 1) | clk=1 sel=0 D=0 -> Q=1  
 30000 ns | prep: hold, D=1 | clk=0 sel=0 D=1 -> Q=1  
 35000 ns | after clk↑ (Q should still be 1) | clk=1 sel=0 D=1 -> Q=1  
 40000 ns | prep: load 0 | clk=0 sel=1 D=0 -> Q=1  
 45000 ns | after clk↑ (Q should be 0) | clk=1 sel=1 D=0 -> Q=1  
 50000 ns | prep: random | clk=0 sel=1 D=0 -> Q=0  
 55000 ns | after clk↑ | clk=1 sel=1 D=0 -> Q=0  
 60000 ns | prep: random | clk=0 sel=1 D=1 -> Q=0  
 65000 ns | after clk↑ | clk=1 sel=1 D=1 -> Q=0  
 70000 ns | prep: random | clk=0 sel=1 D=1 -> Q=1  
 75000 ns | after clk↑ | clk=1 sel=1 D=1 -> Q=1  
 80000 ns | prep: random | clk=0 sel=0 D=1 -> Q=1

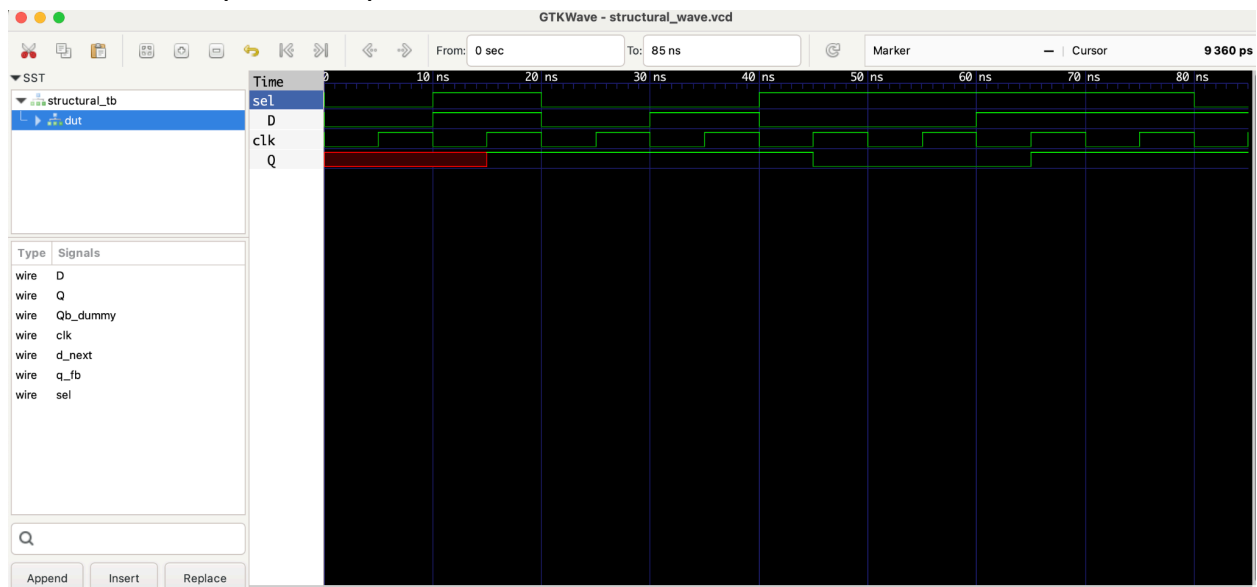
**85000 ns | after clk↑ | clk=1 sel=0 D=1 -> Q=1**

**Logic:**  $Q = (D\_FF\_Output * sel)$

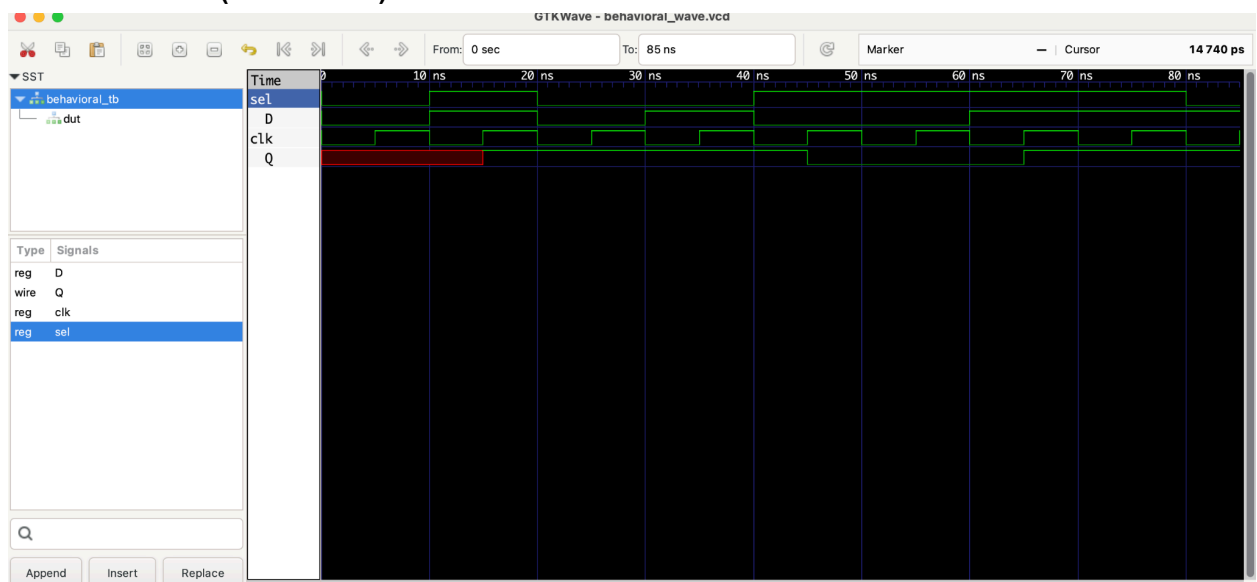
**Explanation: (assuming y is previous output of register)**

A one-bit register is essentially a flip flop with output Q feeding into a 2:1 MUX, and then the MUX's output feeds into the D of the flip flop (flip flop with load register). First, we input (D=y, Clk=Clk) into the D\_FF, obtaining the current FF output, Q. Then, we take our current Q, and input (a=Q, b=D, sel=sel) into the 2:1 MUX, which only loads D if sel is true. If sel is 0, the FF reloads its own Q, holding its previous value.

**Waveform Test (Structural):**



**Waveform Test (Behavioral):**





## Design Code (Structural): \*using D\_latch\_structural module from Part 1

```
/*
D reg = 2:1 MUX preceding D_FF
2 latches in series w complementary enables
    Master: enabled when clk=0 (captures D)
    Slave: enabled when clk=1 (outputs master output)
    2:1 MUX: next_D= D if sel=1, otherwise = prev Q
*/

`timescale 1ps/1ps

//mux w primitive gates
//sel=1 at rising edge ---> Q loads new D
//sel=1 at falling edge ---> Q no change
module mux2_struct(input a, b, sel, output y);
    wire nsel, t0, t1;
    not(nsel, sel);
    //and is transparent when sel=1 or nsel=1
    and(t0, a, nsel);
    and(t1, b, sel);
    //checking for any 1's
    or(y, t0, t1);
endmodule

//Q=w4, Qb=W3
module D_FF_structural(input D, clk, output Q, Qb);
    wire clk_n;
    //need wire for not clk for slave (second circuit)
    not(clk_n, clk);
    wire Qm, Qmb; //master latch outputs
    //master latch on when clk=0, D=x -> 0 is noncontrolling for NAND
    D_latch_structural master(.D(D), .clk(clk_n), .Q(Qm), .Qb(Qmb));
    D_latch_structural slave(.D(Qm), .clk(clk), .Q(Q), .Qb(Qb));
endmodule

//final output based on sel
//if sel=1 at rising edge -> feed in external D
//if sel=0 at risin edge -> feed current Q back in so no change
module OBR_structural(input D, sel, clk, output Q);
    wire q_fb; //feedback from FF output
    wire d_next;
```

```

    wire Qb_dummy;

    mux2_struct u_mux(.a(q_fb), .b(D), .sel(sel), .y(d_next));
    //D_next = q_fb or D
    D_FF_structural u_ff(.D(d_next), .clk(clk), .Q(q_fb), .Qb(Qb_dummy));

    assign Q = q_fb; //solders Q output to q_fb internal wire
endmodule

```

### Design Code (Behavioral):

```

`timescale 1ps/1ps
`default_nettype wire

//1-it reg w load enable
//sel = 1 at posedge clk -> load D
//sel -0 at posedge clk -> hold prev Q

module OBR_behavioral(
input D, sel, clk,
output reg Q
);
    always @(posedge clk) begin
        if(sel)
            Q <=D;
        //else (implied)
        //Q <= Q;
    end
endmodule

```

### Testbench Code (Structural):

```

`timescale 1ns/1ps

module structural_tb;

    reg clk = 0;
    reg D    = 0;
    reg sel = 0;
    wire Q;

    // instantiate DUT

```

```

OBR_structural dut (
    .D(D),
    .sel(sel),
    .clk(clk),
    .Q(Q)
);

// clock: 10 ns period
always #5 clk = ~clk;

// pretty print task
task show(input string tag);
    $display("%0t ns | %s | clk=%0b sel=%0b D=%0b -> Q=%0b",
        $time, tag, clk, sel, D, Q);
endtask

initial begin
    $dumpfile("structural_wave.vcd");
    $dumpvars(0, structural_tb);

    // time 0
    show("start (Q may be x until first edge)");

    // 1) load 1
    @(negedge clk); D = 1; sel = 1; show("prep: load 1");
    @(posedge clk);      show("after clk; (Q should be 1)");

    // 2) hold (sel=0) even if D changes
    @(negedge clk); sel = 0; D = 0; show("prep: hold, D=0");
    @(posedge clk);      show("after clk; (Q should still be 1)");
    @(negedge clk); sel = 0; D = 1; show("prep: hold, D=1");
    @(posedge clk);      show("after clk; (Q should still be 1)");

    // 3) load 0
    @(negedge clk); sel = 1; D = 0; show("prep: load 0");
    @(posedge clk);      show("after clk; (Q should be 0)");

    // 4) a few random cycles
    repeat (4) begin
        @(negedge clk);
        D = $random;
        sel = $random;
    end
end

```

```
        show("prep: random");  
        @(posedge clk);  
        show("after clk");  
    end  
  
    $display("TB done.");  
    $finish;  
end  
endmodule
```